

Relatório de Auditoria de Segurança

ContabOne (monorepo contab-one)

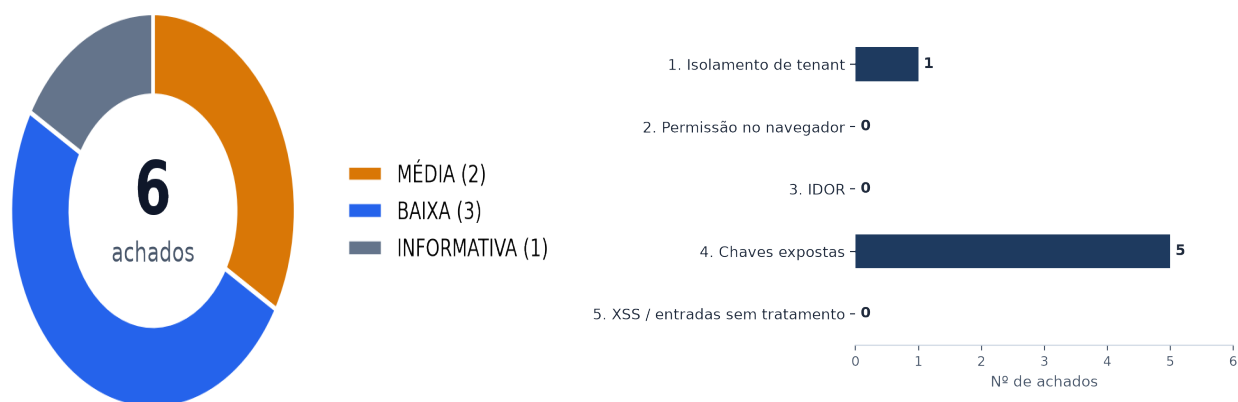
Data do relatório	28 de agosto de 2026
Escopo auditado	ContabOne.Api (.NET 10 / ASP.NET Core minimal APIs + EF Core + PostgreSQL), ContabOne.Frontend (Vue 3 + TypeScript + Pinia + PrimeVue), Det.Agent e Nfse.Agent (automações Python), arquivos de deploy (docker-compose, railway.json, .gitignore) e histórico do git
Categorias avaliadas	5 — isolamento de tenant, permissão no navegador, IDOR, chaves expostas, XSS/entradas sem tratamento
Achados	6 (0 crítica, 0 alta, 2 média, 3 baixa, 1 informativa)
Metodologia	Revisão manual, arquivo por arquivo, de 100% dos handlers de rota do backend (13 arquivos de Features, ~55 arquivos .cs), do frontend (70 arquivos .vue/.ts) e dos artefatos de configuração/deploy — sem amostragem

Nota metodológica — mapeamento das categorias para a stack detectada

O projeto **não usa Supabase/RLS**: é uma API própria em ASP.NET Core com EF Core sobre PostgreSQL, autenticação via ASP.NET Identity + JWT (usuários web) e uma chave de API dedicada para agentes de campo (X-API-Key). Cada categoria da auditoria foi adaptada a este contexto: **(1) isolamento de tenant** foi verificado como os Global Query Filters do EF Core em AppDbContext.cs, populados por um TenantContext resolvido em middleware a partir de claims do JWT/API key; **(2) permissão no navegador** foi verificada cruzando os papéis exigidos pelas rotas do Vue Router com as policies de autorização (RequireAuthorization) de cada grupo de endpoints no backend; **(3) IDOR** foi verificado em cada handler que busca/altera/remove objeto por Guid, considerando que FindAsync/consultas LINQ do EF Core aplicam os Global Query Filters automaticamente (comportamento confirmado) — exceto onde IgnoreQueryFilters() é usado explicitamente, cada ocorrência das quais foi auditada individualmente; **(4) chaves expostas** cobriu código-fonte, appsettings, docker-compose, railway.json, arquivos de exemplo dos agentes Python e o histórico completo do git; **(5) XSS** cobriu os equivalentes Vue (v-html) e bindings dinâmicos de href/src, já que o frontend é Vue 3, não React.

Resumo executivo

Foram identificados **6 achados** nas 5 categorias avaliadas, todos de severidade média, baixa ou informativa — **nenhum achado crítico ou alto**. As categorias **isolamento de tenant** (fora do achado pontual sobre um segredo compartilhado), **permissão definida no navegador**, **IDOR** e **XSS** foram revisadas integralmente sem vulnerabilidade explorável encontrada — o backend aplica controles equivalentes ou mais restritivos que a UI em todos os casos verificados. Os achados concentram-se em **chaves/segredos expostos**, principalmente ligados a ambiente de desenvolvimento.



Achados por severidade

Achados por categoria

Categoria	Achados	Situação
1. Isolamento de tenant	1	1 achado(s) — ver detalhamento
2. Permissão no navegador	0	Revisado — sem vulnerabilidade explorável
3. IDOR	0	Revisado — sem vulnerabilidade explorável
4. Chaves expostas	5	5 achado(s) — ver detalhamento
5. XSS / entradas sem tratamento	0	Revisado — sem vulnerabilidade explorável

Pontos fortes

Controles verificados no código real e confirmados como corretos, com a respectiva evidência. Esta seção documenta a cobertura da auditoria tanto quanto os controles em si.

✓ Isolamento multi-tenant fail-closed via Global Query Filters do EF Core

ContabOne.Api/Infra/AppDbContext.cs:45-69

Todas as entidades sensíveis (Agente, Cliente, Execucao, ExecucaoMetrica, ConfiguracaoEscritorio, Alerta, EscritorioProduto) têm o filtro ``VeTodosOsEscritorios || x.Escritoriold == tenant.Escritoriold``. Quando nem admin nem escritório estão resolvidos, o filtro vira ``Escritoriold == null``, que nunca combina (a coluna é Guid não-nulo) — resultando em zero linhas. O padrão é fail-closed, não fail-open.

✓ Middleware rejeita sessão sem escritório identificável em vez de abrir o filtro

ContabOne.Api/Infra/TenantContextMiddleware.cs:37-56

Um usuário com papel de escritório mas sem escritorio_id resolvível no token recebe 401 explícito, em vez de seguir com TenantContext vazio (o que abriria a condição ``Escritoriold == null``).

✓ Todos os usos de IgnoreQueryFilters() auditados e legítimos

27 ocorrências em 8 arquivos (ApiKeyAuthenticationHandler, AdminEndpoints, AgentEndpoints, AgentesManagementEndpoints, ProdutosEndpoints, UsuariosEndpoints, SeedEndpoints, Jobs/AlertaJob)

Cada ocorrência foi verificada individualmente: lookups pré-autenticação (antes do TenantContext existir), endpoints exclusivos de PlatformAdmin, ou o cron job iterando explicitamente por escritório. Nenhum uso encontrado que vazze dados entre tenants para um usuário comum.

✓ Escalação de privilégio bloqueada na gestão de usuários

ContabOne.Api/Features/Usuarios/UsuariosEndpoints.cs:239-258 (ResolverEscopo)

O escritório de um usuário criado/editado por um EscritorioAdmin vem sempre de tenant.Escritoriold (token), nunca do corpo da requisição. Só PlatformAdmin pode conceder o papel PlatformAdmin ou mover um usuário para outro escritório. Um usuário não pode alterar o próprio papel nem desativar a si mesmo.

✓ IDOR mitigado por busca escopada ao tenant, com 404 em vez de 403

UsuariosEndpoints.cs:220-232 (BuscarNoEscopoAsync), TourEndpoints.cs:22-33 (UsuarioId sempre do token), ProdutosEndpoints.cs:34-38 (comentário cita explicitamente "seria IDOR (\$5)")

Objeto de outro tenant não é distinguível de objeto inexistente (404 em ambos os casos), evitando confirmar a existência de recursos de terceiros.

✓ Paridade completa entre papéis exigidos no frontend e políticas de autorização no backend

ContabOne.Frontend/src/router/index.ts (meta.papeis) × ContabOne.Api/Program.cs (RequireAuthorization)

Cada uma das 11 rotas protegidas do frontend foi cruzada manualmente com a policy do grupo de endpoints correspondente no backend. Nenhuma divergência encontrada — toda operação que o frontend esconde por papel é também rejeitada pelo servidor caso chamada diretamente.

✓ JWT com validação completa dos dois lados e cookies de sessão bem configurados

ContabOne.Api/Program.cs:102-119, Features/Auth/AuthEndpoints.cs:84-91

Issuer, audience, tempo de vida e assinatura validados; emissão e validação leem a mesma configuração (evita rejeitar os próprios tokens). Refresh token em cookie HttpOnly+Secure+SameSite=Strict; access token de vida curta (15 min) fica em sessionStorage do frontend, não em localStorage.

✓ Login resistente a enumeração de usuários por tempo de resposta

ContabOne.Api/Features/Auth/AuthEndpoints.cs:28-53

Um hash "descartável" é verificado quando o e-mail não existe, gastando o mesmo tempo de derivação de chave (~90ms) de uma verificação real — a resposta não denuncia por timing quais e-mails existem na base.

✓ Único uso de v-html do frontend é seguro por construção

ContabOne.Frontend/src/components/comum/IconeCatalogo.vue:12, 26-36

Renderiza sempre um de três SVGs fixos definidos no próprio componente, selecionados por chave. O valor vindo do banco (Produto.Icone) é usado só como CHAVE de lookup, nunca interpretado como HTML — um valor desconhecido cai no ícone genérico. Documentado explicitamente no próprio código.

✓ Nenhuma outra superfície de XSS encontrada no frontend

Busca em todo ContabOne.Frontend/src (70 arquivos .vue/.ts)

Nenhum innerHTML, dangerouslySetInnerHTML equivalente, eval(), new Function(), ou biblioteca de renderização de Markdown/HTML de terceiros (confirmado via package.json). Os únicos bindings :src encontrados apontam para assets estáticos importados (logos), não para dado controlado por usuário.

✓ Segredos reais nunca versionados no repositório

.gitignore (seções 1 e 4) + varredura do histórico completo do git

Cobertura explícita para *.pfx, *.p12, *.pem, *.key, config.toml, .env, _agente_cache.json, certificados/, notas/. Busca em todo o histórico do git (`git log --all --diff-filter=A`) não encontrou nenhum desses arquivos jamais commitado.

✓ Criptografia bem projetada para o blob de configuração do handshake do agente

ContabOne.Api/Security/ConfiguracaoCipher.cs:18-50

AES-256-GCM com nonce aleatório de 12 bytes por operação e chave derivada por HMAC-SHA256 da própria API key do agente — não introduz nenhum segredo novo em variáveis de ambiente.

✓ Cobertura de teste automatizado dedicada ao isolamento multi-tenant

ContabOne.Api.Tests/IsolamentoTest.cs

Suíte de integração testa explicitamente que um agente (via X-API-Key) ou usuário (via JWT) de um escritório não alcança dados de outro, inclusive tentando furar o filtro via parâmetro de query — com o comentário "a quebra disso seria o pior defeito possível neste produto".

✓ CORS fail-closed em produção

ContabOne.Api/Program.cs:142-164

Se a variável CORS_ORIGINS não for configurada em produção, a lista de origens permitidas fica vazia — bloqueando toda requisição cross-origin — em vez de permitir qualquer origem por omissão.

✓ Rate limiting aplicado ao endpoint de login e aos endpoints de agente

ContabOne.Api/Program.cs:175-206, 274-283

Política "auth" (10 req/min por IP) no grupo /api/auth e política "agent" (60 req/min) no grupo /api/agent, com fila e status 429 correto em vez do 503 padrão do ASP.NET Core.

Achados detalhados

Cada achado lista categoria, severidade, localização exata (arquivo:linha), trecho de código, justificativa de explorabilidade e condições necessárias para exploração.

F1 · Credenciais de ambiente de dev versionadas em texto claro na raiz do repositório

MÉDIA**Categoria 4** — Chaves expostas

DadosAcesso.txt:1-15

Trecho:

```
Login admin: admin@nfse.local / Admin123!
Login escritório: escritorio@nfse.local / Admin123!
Login usuario: usuario@nfse.local / Admin123!
```

Por que é explorável:

O arquivo está commitado no git (commit f16576b, "login e senha do ambiente dev") na raiz do repositório. As credenciais correspondem exatamente às contas criadas por POST /api/seed/dev (ContabOne.Api/Features/Seed/SeedEndpoints.cs:137-164). Credenciais reais (mesmo que de um ambiente de desenvolvimento) permanecem no histórico do git indefinidamente, são capturadas por scanners de segredo (GitHub secret scanning, gitleaks) e, se qualquer ambiente compartilhado subir com ASPNETCORE_ENVIRONMENT=Development por engano, dão acesso administrativo (PlatformAdmin) completo e imediato.

Condição de explorabilidade:

Exige que /api/seed/dev tenha sido executado contra o banco alvo, o que só é possível quando IsDevelopment() é verdadeiro (gate duplo em SeedEndpoints.cs).

F2 · Segredos de desenvolvimento hardcoded sem validação de ambiente na inicialização

BAIXA**Categoria 4** — Chaves expostas

ContabOne.Api/appsettings.Development.json:7-9

Trecho:

```
"ConnectionStrings": { "Default": "...;Password=contabone;..." }
"JWT_SIGNING_KEY": "dev-key-change-in-production-min-32-chars!!"
"HMAC_CNPJ_KEY": "dev-hmac-cnpj-key-change-in-production!!"
```

Por que é explorável:

Program.cs (linhas 85-94) exige que JWT_SIGNING_KEY e HMAC_CNPJ_KEY estejam definidos, mas não valida que NÃO sejam estes valores de exemplo conhecidos publicamente no repositório. O arquivo só é carregado quando ASPNETCORE_ENVIRONMENT=Development, mas nada no código impede a API de subir normalmente em produção assinando tokens JWT com esta chave pública caso essa variável seja erroneamente definida como "Development" — permitindo forjar tokens de qualquer papel, inclusive PlatformAdmin.

Condição de explorabilidade:

Requer erro de configuração do ambiente de deploy — não é o comportamento padrão (.NET assume "Production" quando a variável não é definida).

F3 · Senha trivial hardcoded para Postgres local, exposta na porta padrão do host

BAIXA

Categoria 4 — Chaves expostas

docker-compose.yml:8-9, 11-12

Trecho:

```
POSTGRES_USER: contabone
POSTGRES_PASSWORD: contabone
ports:
  - "5432:5432"
```

Por que é explorável:

O bind "5432:5432" expõe a porta em todas as interfaces do host Docker (0.0.0.0), não apenas em loopback. Em uma workstation local atrás de firewall isso é baixo risco; em uma VM de nuvem compartilhada ou ambiente com portas abertas, é uma instância Postgres com credenciais triviais (usuário = senha) acessível pela rede.

Condição de explorabilidade:

Aplica-se apenas ao ambiente local via docker-compose. O deploy real usa DATABASE_URL gerenciado pela plataforma (Railway), não este compose.

F4 · Connection string de fallback hardcoded no factory de design-time do EF Core

INFORMATIVA

Categoria 4 — Chaves expostas

ContabOne.Api/Infra/AppDbContextFactory.cs:17-21

Trecho:

```
rawConnString = Environment.GetEnvironmentVariable("DATABASE_URL");
if (string.IsNullOrEmpty(rawConnString))
    rawConnString = "Host=localhost;...;Password=contabone;...";
```

Por que é explorável:

Mesmas credenciais triviais do docker-compose, porém este código só executa durante "dotnet ef migrations" (ferramenta de design-time), nunca no processo da API publicado — Program.cs exige DATABASE_URL/ConnectionStrings:Default explicitamente e lança exceção se ausente. Risco teórico, não é um vetor de ataque em produção.

Condição de explorabilidade:

Só é executado localmente por um desenvolvedor rodando o CLI do EF Core.

F5 · Endpoints de seed sem segundo fator além do gate de ambiente (IsDevelopment)

BAIXA

Categoria 4 — Chaves expostas

ContabOne.Api/Features/Seed/SeedEndpoints.cs:17-79

Trecho:

```
app.MapPost("/api/seed/admin", async (SeedAdminRequest req, ...) => {
    if (!env.IsDevelopment()) return Results.NotFound();
    ... Papel = PapelUsuario.PlatformAdmin ...
}).AllowAnonymous();
```

Por que é explorável:

Gate duplo — não mapeado fora de Development (Program.cs:266-271) e cada handler reconfirma env.IsDevelopment() — mas o único controle é a variável ASPNETCORE_ENVIRONMENT. Não há um segundo fator (ex.: token comparado a um header) que sobreviva a uma configuração incorreta dessa variável. Se ela for setada como "Development" em qualquer ambiente real, qualquer visitante anônimo cria uma conta PlatformAdmin com senha arbitrária via /api/seed/admin, e /api/seed/status despeja e-mail/papel/escritório de todos os usuários via SQL bruto.

Condição de explorabilidade:

Requer erro de configuração de ambiente; não reproduzido em produção.

F6 · Segredo HMAC_CNPJ_KEY global compartilhado devolvido a todo agente de campo autenticado

MÉDIA**Categoria 1** — Isolamento de tenant

ContabOne.Api/Features/Agent/AgentEndpoints.cs:94-124

Trecho:

```
return Results.Ok(new HandshakeResponse {  
    ...  
    HmacCnpjKey = config["HMAC_CNPJ_KEY"]!, // same for all agents of this escritorio  
    ...  
});
```

Por que é explorável:

O mecanismo de isolamento do ContabOne é por-tenant via Global Query Filters do EF Core (AppDbContext.cs), mas este segredo criptográfico é ÚNICO PARA TODA A PLATAFORMA, não escopado por escritório. A cada handshake bem-sucedido, a API devolve o HMAC_CNPJ_KEY em texto claro a qualquer agente autenticado — um executável Python rodando na máquina de um escritório contábil, com a própria chave de API guardada em config.toml local em texto claro (ver Nfse.Agent/CLAUDE.md, seção "Sensitive local data"). Esse é o mesmo segredo usado para gerar Escritorio.CnpjHash (índice único) e Cliente.CnpjHash de TODOS os escritórios da plataforma. Comprometer um único agente de campo — o elo mais exposto do sistema, por rodar fora do perímetro do servidor — expõe o segredo que permite calcular o hash determinístico de qualquer CNPJ, quebrando a garantia de isolamento por tenant no nível criptográfico mesmo que os filtros de banco continuem corretos.

Condição de explorabilidade:

Mitigante: não foi encontrado, na superfície de API revisada, um endpoint que devolva CnpjHash bruto para comparação entre tenants — a exploração prática além da simples posse do segredo exigiria outro vetor (acesso direto ao banco, por exemplo). Por isso a severidade é MÉDIA, não ALTA.

Recomendações priorizadas

Prior.	Recomendação	Achados
P1	Remover DadosAcesso.txt do repositório (git rm + reescrita do histórico se o repositório for considerado sensível) e mover a instrução de seed para o README, sem credenciais reais em texto claro.	F1
P1	Adicionar uma verificação de startup em Program.cs que recuse subir se JWT_SIGNING_KEY ou HMAC_CNPJ_KEY baterem com os literais de appsettings.Development.json e IsDevelopment() for falso — defesa em profundidade contra erro de configuração de ambiente.	F2, F5
P2	Adicionar um SEED_TOKEN (header ou variável de ambiente comparada a um valor) exigido pelos endpoints /api/seed/*, além do gate de IsDevelopment(), como segunda camada independente da variável de ambiente.	F5
P2	Escopar o segredo de hashing de CNPJ por escritório (ou parar de devolver a chave mestre para o agente), derivando uma chave por-tenant a partir do segredo global — mesmo padrão já usado em ConfiguracaoCipher para a chave de configuração.	F6
P3	Trocar a senha do Postgres do docker-compose.yml por um valor gerado (ou ao menos restringir o bind de porta a 127.0.0.1:5432 em vez de 0.0.0.0:5432) para reduzir a superfície em ambientes de nuvem compartilhados.	F3
P3	Adicionar um teste automatizado que falhe o build se algum valor literal dos segredos de appsettings.Development.json for detectado configurado fora de um ambiente de Development.	F2, F4

Issues para o GitHub

Texto completo, pronto para copiar e colar, de cada issue acionável. Achados triviais do mesmo tema foram agrupados numa única issue para não gerar spam.

--- ISSUE 1 ---

[Segurança] Credenciais de ambiente de dev versionadas em texto claro (DadosAcesso.txt)

Labels sugeridas: security, media

Problema

O arquivo `DadosAcesso.txt`, na raiz do repositório, contém e-mail e senha em texto claro de três contas de desenvolvimento (admin, escritório e usuário), commitado no git (commit `f16576b`). As credenciais correspondem exatamente às contas criadas por `POST /api/seed/dev` (`ContabOne.Api/Features/Seed/SeedEndpoints.cs:137-164`).

Por que é explorável

Credenciais reais no histórico do git permanecem lá indefinidamente (mesmo após um `git rm` posterior), são capturadas por scanners de segredo (gerando ruído permanente) e, caso qualquer ambiente compartilhado suba com `ASPNETCORE\_ENVIRONMENT=Development` por engano, dão acesso administrativo (PlatformAdmin) completo e imediato a quem tiver acesso de leitura ao repositório.

Evidência

```
`DadosAcesso.txt:1-15`  
...
```

```
Login admin: admin@nfse.local / Admin123!  
Login escritório: escritorio@nfse.local / Admin123!  
Login usuario: usuario@nfse.local / Admin123!  
...
```

Impacto

Médio – exige um erro de configuração de ambiente adicional para ser explorável diretamente, mas o hábito de commitar credenciais reais (mesmo de dev) é uma prática de risco permanente no histórico do repositório.

Sugestão de correção

- Remover o arquivo do controle de versão (`git rm DadosAcesso.txt`) e adicioná-lo ao `.gitignore`.
- Mover a lista de credenciais de exemplo para o `README.md`, deixando claro que só existem após rodar `POST /api/seed/dev` localmente.
- Se o repositório for público ou tiver múltiplos colaboradores externos, considerar reescrever o histórico para remover o arquivo (`git filter-repo` ou BFG).

Critérios de aceite

- [] `DadosAcesso.txt` removido do working tree e do índice do git
- [] `.gitignore` atualizado para impedir reintrodução do arquivo
- [] Instruções de seed movidas para README.md sem credenciais reais versionadas
- [] Avaliação feita sobre reescrever o histórico do git (documentar decisão, mesmo que seja "não fazer")

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Reforçar defesa em profundidade contra erro de configuração de ambiente (segredos de dev, endpoints de seed e senha padrão do Postgres local)

Labels sugeridas: security, baixa

Problema

Três controles distintos dependem, hoje, unicamente da variável `ASPNETCORE\_ENVIRONMENT` estar correta para não expor risco real:

1. ``ContabOne.Api/appsettings.Development.json:7-9`` guarda ``JWT_SIGNING_KEY``, ``HMAC_CNPJ_KEY`` e a senha do Postgres local como literais conhecidos publicamente no repositório, sem checagem de startup que os rejeite fora de Development.
2. ``ContabOne.Api/Features/Seed/SeedEndpoints.cs:17-79`` expõe ``/api/seed/admin`` (cria PlatformAdmin com senha arbitrária) e ``/api/seed/status`` (despeja todos os usuários via SQL bruto) como ``AllowAnonymous()``, gateados apenas por ``env.IsDevelopment()``.
3. ``docker-compose.yml:8-9,11-12`` usa ``POSTGRES_PASSWORD: contabone`` com a porta 5432 publicada em todas as interfaces do host (``"5432:5432"``), e o mesmo valor aparece hardcoded como fallback de design-time em ``ContabOne.Api/Infra/AppDbContextFactory.cs:20``.

Por que é explorável

Nenhum destes é explorável no comportamento padrão documentado (produção não carrega o appsettings de Development, e o Postgres do compose não é o banco usado em produção). O risco é de configuração incorreta: se ``ASPNETCORE_ENVIRONMENT`` for setado como ``Development`` em qualquer ambiente real (erro comum em PaaS), os itens 1 e 2 se tornam, respectivamente, uma chave de assinatura JWT pública (forja de token de qualquer papel) e um endpoint anônimo de criação de PlatformAdmin.

Evidência

```
`ContabOne.Api/appsettings.Development.json:7-9`
```json
"ConnectionStrings": { "Default": "...;Password=contabone;..." },
"JWT_SIGNING_KEY": "dev-key-change-in-production-min-32-chars!!",
"HMAC_CNPJ_KEY": "dev-hmac-cnpj-key-change-in-production!!"
...

`ContabOne.Api/Features/Seed/SeedEndpoints.cs:51-58`
```csharp
app.MapPost("/api/seed/admin", async (SeedAdminRequest req, ...) => {
    if (!env.IsDevelopment()) return Results.NotFound();
    ...
}).AllowAnonymous();
...

`docker-compose.yml:8-12`
```yaml
POSTGRES_PASSWORD: contabone
ports:
 - "5432:5432"
...

```

#### ## Impacto

Baixo isoladamente (todos exigem uma pré-condição de configuração incorreta), mas os três compartilham a mesma causa raiz – ausência de defesa em profundidade além de uma única variável de ambiente – e por isso foram agrupados.

#### ## Sugestão de correção

- Adicionar checagem de startup em ``Program.cs`` que recuse subir se ``JWT_SIGNING_KEY``/``HMAC_CNPJ_KEY`` baterem com os literais de exemplo e ``IsDevelopment()`` for falso.
- Exigir um ``SEED_TOKEN`` (header comparado a uma variável de ambiente) nos endpoints ``/api/seed/*``, além do gate de ambiente.
- Trocar a senha do Postgres local por um valor gerado e restringir o bind de porta a ``127.0.0.1:5432``.

#### ## Critérios de aceite

- [ ] Startup falha explicitamente se segredos de exemplo forem detectados fora de Development
- [ ] ``/api/seed/*`` exige um token adicional configurável, não só ``IsDevelopment()``
- [ ] ``docker-compose.yml`` usa senha gerada (ou variável sem default) e porta restrita a loopback
- [ ] ``AppDbContextFactory.cs`` não depende mais de um literal de senha (usar variável obrigatória ou arquivo ``.env`` local ignorado pelo git)

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

## [Segurança] HMAC\_CNPJ\_KEY global devolvido em texto claro a todo agente autenticado

*Labels sugeridas: security, media*

### ## Problema

`ContabOne.Api/Features/Agent/AgentEndpoints.cs:122` devolve `HMAC\_CNPJ\_KEY` – uma chave única para toda a plataforma – em texto claro no corpo da resposta de `/api/agent/handshake`, para qualquer agente de campo que se autentique com sucesso via `X-API-Key`.

### ## Por que é explorável

O mecanismo de isolamento por-tenant do ContabOne é feito via Global Query Filters do EF Core (`AppDbContext.cs`), mas este segredo criptográfico específico não é escopado por escritório – é o mesmo para todos. Um agente de campo é um executável Python rodando em máquinas de terceiros (escritórios contábeis), com a própria API key guardada em `config.toml` local em texto claro (ver `Nfse.Agent/CLAUDE.md`, seção "Sensitive local data"). Esse é objetivamente o componente mais exposto do sistema. Comprometer um único agente expõe o segredo usado para gerar `Escritorio.CnpjHash` e `Cliente.CnpjHash` de TODOS os escritórios da plataforma, permitindo calcular o hash determinístico de qualquer CNPJ e testar sua presença na base – uma quebra da garantia de isolamento por tenant no nível criptográfico, mesmo que os filtros de banco permaneçam corretos.

### ## Evidência

```
`ContabOne.Api/Features/Agent/AgentEndpoints.cs:105-124`
``csharp
return Results.Ok(new HandshakeResponse {
 ...
 HmacCnpjKey = config["HMAC_CNPJ_KEY"]!, // same for all agents of this escritorio
($PLANO_SAAS_AGENTE §6)
 ConfiguracaoCifrada = configuracaoCifrada,
});
```  
  
Consumido em `ContabOne.Api/Security/CnpjHasher.cs:15-20` (`HMACSHA256` sobre o CNPJ limpo).
```

Impacto

Médio – não foi encontrado, na superfície de API revisada, um endpoint que aceite comparar/consultar `CnpjHash` bruto entre tenants, o que limita a exploração prática imediata. Ainda assim, é uma violação do princípio de segredo por-tenant que aumenta o raio de explosão de qualquer comprometimento de agente.

Sugestão de correção

- Derivar uma chave de hashing de CNPJ por-escritório a partir do segredo mestre (ex.: `HMAC-SHA256(key=HMAC\_CNPJ\_KEY\_mestre, msg=escritorioId)`), no mesmo espírito do que já é feito em `ConfiguracaoCipher` para a chave de configuração – e devolver ao agente apenas a chave derivada do seu próprio escritório.
- Avaliar se o hashing de CNPJ pode ser movido inteiramente para o servidor (o agente envia o CNPJ mascarado + hash calculado localmente hoje) para eliminar a necessidade de distribuir qualquer segredo de hashing a um processo fora do perímetro do servidor.

Critérios de aceite

- [] Chave de hashing de CNPJ devolvida ao agente é específica do escritório dele, não a chave mestre
- [] Migração de dados existentes (CnpjHash já gravados com a chave antiga) planejada e documentada
- [] Teste de regressão comprovando que a chave de um escritório não permite recalcular hash válido para outro escritório

--- FIM ISSUE 3 ---